

VIETNAM NATIONAL UNIVERSITY HO CHI MINH CITY
HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY
FACULTY OF COMPUTER SCIENCE AND ENGINEERING



DEEP LEARNING AND ITS APPLICATIONS

CO3133 - Semester 261

ASSIGNMENT 1 - M1 DRAFT

Group G-M10

Fashion-MNIST: Linear Classifier and MLP

Advisor(s): Lê Thành Sách

Student(s): Trần Gia Lâm (Leader) 2352670

Nguyễn Hữu Cầu 2352129

HO CHI MINH CITY, SEPTEMBER 2026



Team and draft scope

Responsibilities - Group G-M10

Member	Student ID	Role	Responsibility
Trần Gia Lâm	2352670	Leader	Ran the Google Colab Tesla T4 experiments; performed result and error analysis; verified the experimental outputs; integrated and prepared the report.
Nguyễn Hữu Cầu	2352129	Member	Developed and reviewed the EDA and MLP components.

Scope. This M1 draft covers EDA, Dataset/DataLoader setup, the training and validation loop, and two Fashion-MNIST baselines: a Linear Classifier and an MLP. It is organized into three parts: problem and data; methodology and experimental design; results and discussion.

Results. Both models were trained for 10 epochs on a Google Colab Tesla T4, using the same 54,000 training images and 6,000 validation images with seed 42. The reported results come from `outputs/a1_t4/`. The test set has not been evaluated.

Code and supporting files.

- Repository: <https://github.com/cau26/C03133-DLProject>.
- [Original Colab notebook for the T4 run](#).
- [Run provenance and verification](#).
- [AI Usage Disclosure](#).



Contents

1	Problem and Data	1
1.1	Problem statement	1
1.2	Data source and split	1
1.3	Exploratory data analysis	3
1.4	Preprocessing, Dataset and DataLoader	4
2	Methodology and Experimental Design	6
2.1	Pipeline and model architectures	6
2.2	Training, validation and checkpoint selection	7
2.3	Hypothesis, controlled variables and metrics	8
3	Results and Discussion	9
3.1	Results at the selected checkpoints	9
3.2	Learning curves	9
3.3	Confusion matrices and class-level errors	11
3.4	Correct and incorrect predictions	14
3.5	Computational cost and timing scope	15
3.6	Implementation issues	16
3.7	Reproduction and provenance	16
3.8	Conclusions, limitations and next steps	17
	AI Usage Disclosure	18
	References	18



1 Problem and Data

1.1 Problem statement

Given a grayscale image of a fashion item, the model predicts one of the 10 Fashion-MNIST classes. Each input has shape (1, 28, 28), and its label is an integer from 0 to 9. The model produces 10 logits; the class with the largest logit is the prediction.

We want to build a reproducible pipeline and test whether an MLP with one hidden layer performs better than a linear classifier under the same data split and training protocol. This is a benchmark experiment. Performance on real product photographs has not been tested.

1.2 Data source and split

Fashion-MNIST is published by Zalando Research under the MIT license [1]. It contains 60,000 official training images and 10,000 official test images. Each image is 28 x 28 pixels with one channel. Our code uses `torchvision.datasets.FashionMNIST` and downloads the gzip-compressed IDX files from the authors' HTTPS endpoint. Torchvision checks the expected MD5 checksums when downloading them.

The images and 10 class labels are used as released. We do not relabel the data. The four MD5 checksums in `data_summary.json` identify the dataset files; the run does not record a separate release version. The four compressed files total 30,878,645 bytes (about 30.88 MB), or 54,950,048 bytes (about 54.95 MB) after decompression. These sizes refer to the verified dataset files, excluding the Colab environment, libraries and model checkpoints.

We split the official training set with `train_test_split`, stratify by class label, set `validation_size` to 6,000 and use seed 42. Both models use the same 54,000 training indices and 6,000 validation indices. The official test set is reserved. Its class counts are included in EDA, but it is not used for model evaluation or hyperparameter selection.



Table 1: Class counts in the training, validation and official test sets.

Class	Training	Validation	Official test
T-shirt/top	5,400	600	1,000
Trouser	5,400	600	1,000
Pullover	5,400	600	1,000
Dress	5,400	600	1,000
Coat	5,400	600	1,000
Sandal	5,400	600	1,000
Shirt	5,400	600	1,000
Sneaker	5,400	600	1,000
Bag	5,400	600	1,000
Ankle boot	5,400	600	1,000

Total: 54,000 training, 6,000 validation and 10,000 test images. The indices are saved in [split_indices.npz](#). The code checks that the training and validation indices do not overlap and together cover all 60,000 official training samples. Image-content duplicates have not been checked, so disjoint indices alone do not rule out visually identical images across the two subsets.

The data source, expected checksums and split fingerprint are recorded in [data_summary.json](#).

1.3 Exploratory data analysis

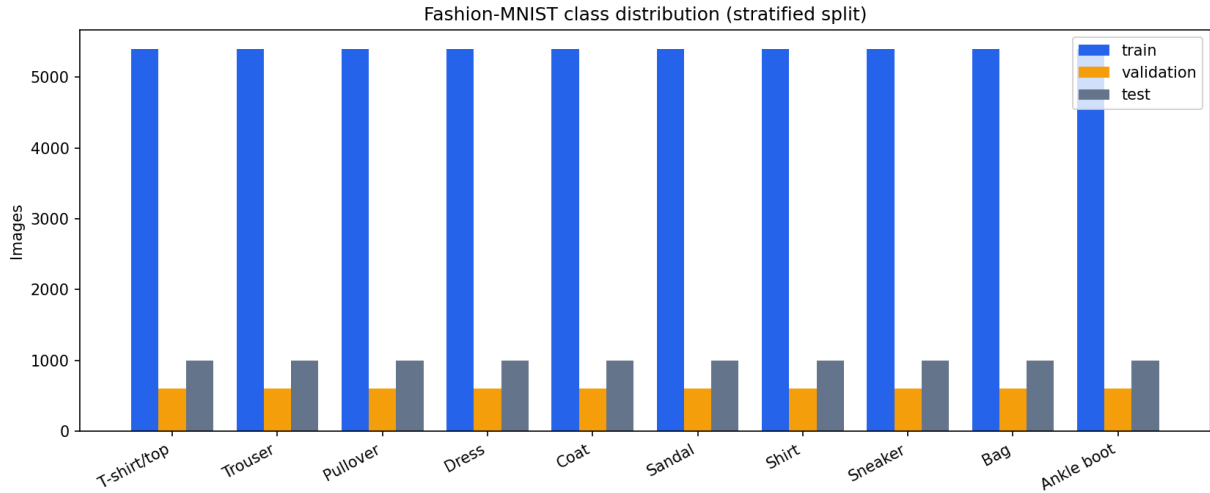


Figure 1: Class distribution across the dataset splits

Training and validation are balanced by class. The largest-to-smallest training class ratio is 1.0, so these baselines use no class weighting, oversampling or undersampling. Equal class counts do not imply equal classification difficulty.



Figure 2: The first two training images selected for each class

T-shirt/top, Shirt, Pullover and Coat can share similar outlines and sleeve lengths. At 28×28 pixels, grayscale images retain limited detail about collars, folds and fabric texture. These similarities may help explain some of the confusion patterns in Part 3. They are visual observations, rather than evidence of which features the models actually use.

1.4 Preprocessing, Dataset and DataLoader

ToTensor converts uint8 pixel values in $[0, 255]$ to float32 values in $[0, 1]$. We apply no additional mean/std normalization, data augmentation or pretrained feature extraction.

All 60,000 official training images are retained when creating the training and validation subsets. There is no extra sample filtering, label correction or dataset-specific cleaning rule. Images remain 28×28 . Both the one-epoch pipeline check and the main experiment use Fashion-MNIST; MNIST and CIFAR-10 were not used in this draft.

We use torchvision's FashionMNIST Dataset, Subset for the saved indices, and PyTorch's DataLoader for batching. The split and loader setup is in `a1/data.py`. The underlying Dataset is provided by torchvision.

The inspected batch has image shape $(128, 1, 28, 28)$, label shape $(128,)$, float32 images, int64 labels and pixel values in $[0, 1]$. The training loader shuffles with a generator seeded at 42. The validation loader does not shuffle. We use `num_workers = 0` and keep the final, smaller batch.

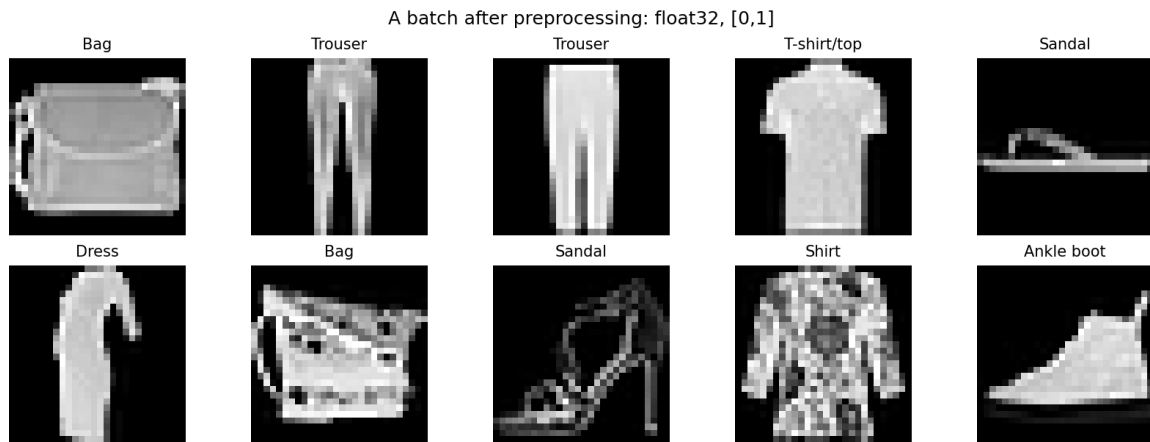


Figure 3: A training batch after ToTensor

2 Methodology and Experimental Design

2.1 Pipeline and model architectures

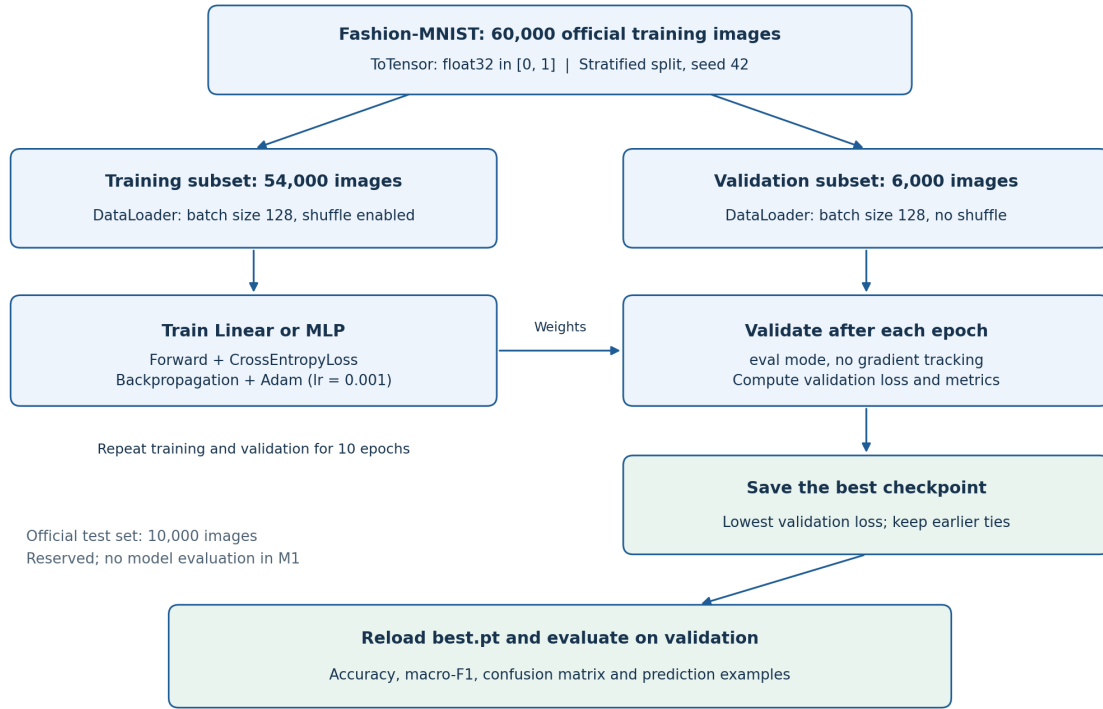


Figure 4: Pipeline used by the Linear and MLP baselines

The diagram describes the implementation. All measured results come from `output s/a1_t4/`. The test set is outside the training and checkpoint-selection process.

Table 2: Model architectures and trainable parameter counts.

Model	Architecture	Parameters
Linear	Flatten → Linear(784, 10) → logits	7,850
MLP	Flatten → Linear(784, 256) → ReLU → Linear(256, 10) → logits	203,530

The Linear model has $784 \times 10 + 10 = 7,850$ parameters. The MLP has $(784 \times 256 + 256) + (256 \times 10 + 10) = 203,530$ parameters.

The Linear model learns linear decision boundaries over flattened pixels. The MLP adds a hidden layer and ReLU to represent nonlinear relationships. Both models use flattened images. Unlike a CNN, neither architecture directly imposes local spatial connections or convolutional weight sharing. Flattening preserves the pixel values, but these architectures do not explicitly model the 2D neighborhood structure.

ReLU applies $\max(0, z)$ to each element: negative values become zero and positive values pass through. This activation introduces nonlinearity between the two Linear layers. Without an intervening nonlinear operation, two stacked Linear layers are equivalent to one affine transformation. The current experiment uses no dropout or weight decay.

Both models return logits directly to CrossEntropyLoss, with no Softmax before the loss. Predictions use the argmax of the logits.

2.2 Training, validation and checkpoint selection

Table 3: Training configuration and recorded runtime environment.

Setting	Value
Dataset	Fashion-MNIST
Seed / split	42; stratified 54,000 training / 6,000 validation
Batch size / epochs	128 / 10
Optimizer / learning rate	Adam / 0.001
Loss	CrossEntropyLoss
MLP hidden size	256
Hardware	Google Colab, Tesla T4, CUDA
PyTorch CPU threads / loader workers	4 / 0
Python / torch / torchvision	3.13.15 / 2.8.0+cu128 / 0.23.0
numpy / scikit-learn / matplotlib	2.3.5 / 1.8.0 / 3.10.8
Scheduler / dropout / weight decay	Not used
Early stopping / mixed precision	Not used
Checkpoint rule	Lowest validation loss; keep the earlier checkpoint if losses are equal



The notebook prints the GPU name and PyTorch version; each model's metrics.json records the full environment. The models run on the T4 through CUDA. CPU threads control only PyTorch's CPU work.

Each training epoch uses train mode: clear gradients, compute logits and loss, run backpropagation, then call optimizer.step. Epoch loss is weighted by batch size, including the smaller final batch. Gradients are calculated only during training.

Validation runs after every epoch in eval mode, without gradient tracking or parameter updates. When validation loss improves, the code saves best.pt. After 10 epochs, it reloads the checkpoint to calculate metrics, save predictions, and plot confusion matrices and examples.

Both models use a1/train.py. Before each model, the code resets the seed and recreates the training loader with a seeded generator to keep sample order comparable. It seeds Python, NumPy and PyTorch and disables cuDNN benchmarking. Bitwise agreement across hardware or library versions is not guaranteed.

PyTorch provides layers, autograd, the optimizer and loss. Torchvision supplies the Dataset and ToTensor. Scikit-learn handles splitting and metrics; matplotlib creates plots. The repository connects these components into one pipeline.

2.3 Hypothesis, controlled variables and metrics

Our hypothesis is that an MLP with one hidden layer will achieve higher validation accuracy and macro-F1 than the Linear model under this setup. The architecture changes, while the dataset, split, seed, preprocessing, optimizer, learning rate, loss, batch size, epoch budget and T4 environment stay fixed.

We compare both metrics at the checkpoints selected by validation loss. Parameter counts and timings show the cost of any improvement. The MLP has about 25.9 times as many parameters, so the experiment cannot separate the effect of nonlinearity from the effect of model capacity. We used one seed and one configuration, without hyperparameter search or repeated runs.

Accuracy is the fraction of correct predictions among the 6,000 validation samples. Macro-F1 averages the F1 scores of all 10 classes with equal weight and is reported on a 0-1 scale. Confusion-matrix rows represent true labels and columns represent predictions. A confusion rate is the number of errors for a class pair divided by the 600 images of the true class.

3 Results and Discussion

3.1 Results at the selected checkpoints

Table 4: Validation results at the checkpoints selected by minimum validation loss.

Model	Val loss	Val accuracy	Val macro-F1	Parameters	Selected epoch
LINEAR	0.3980	86.62%	0.8644	7,850	9
MLP	0.2932	89.30%	0.8924	203,530	9

Sources: [comparison.csv](#), [Linear metrics](#) and [MLP metrics](#). Each row uses the same selected checkpoint for all reported metrics.

Linear correctly classifies 5,197 of the 6,000 images and makes 803 errors. MLP correctly classifies 5,358 images and makes 642 errors. In this run, MLP improves accuracy by about 2.68 percentage points and macro-F1 by 0.0280, with about 25.9 times as many parameters. These are observations from one seed; statistical significance has not been tested.

3.2 Learning curves

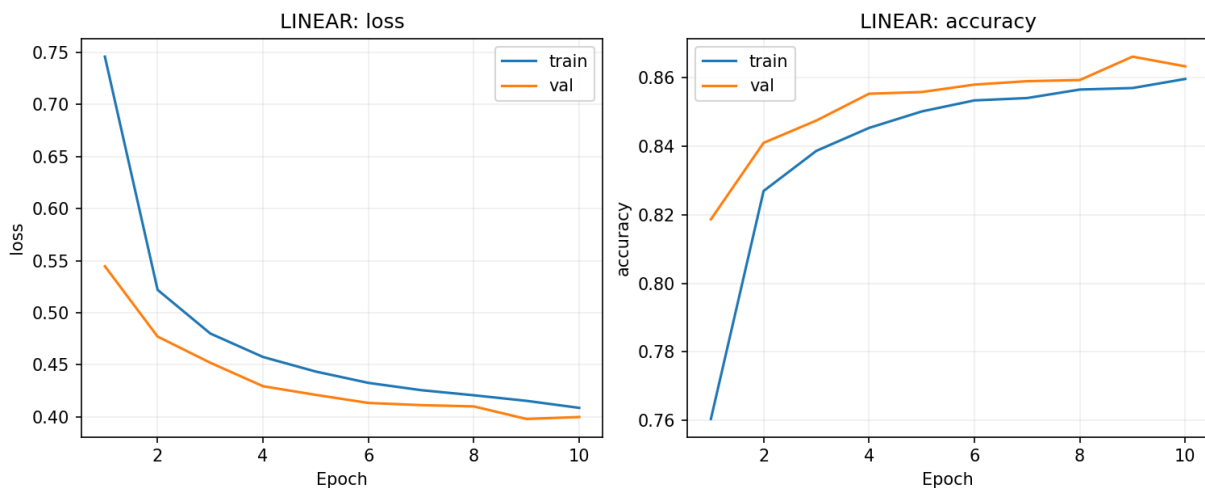


Figure 5: Linear learning curves over 10 epochs

Linear’s validation loss falls from 0.5446 at epoch 1 to 0.3980 at epoch 9. At epoch 10, training loss continues to fall, but validation loss rises slightly to 0.3998 and accuracy drops from 86.62% to 86.33%. The checkpoint rule therefore selects epoch 9. This small final increase is not enough to establish severe overfitting or full convergence.

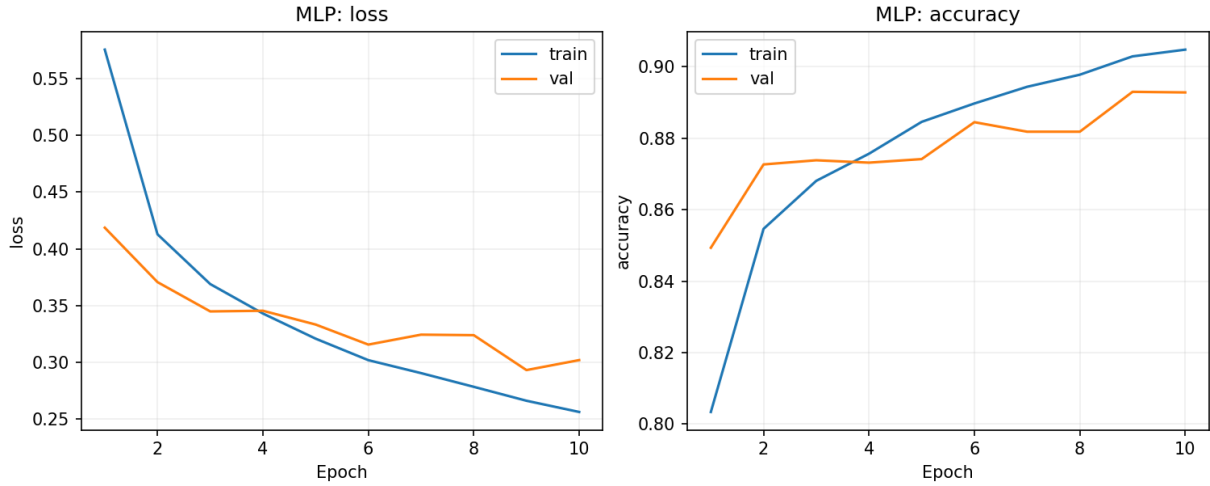


Figure 6: MLP learning curves over 10 epochs

MLP’s training loss falls from 0.5754 to 0.2563. Validation loss fluctuates at epochs 4, 7-8 and 10, reaching its lowest value of 0.2932 at epoch 9. At epoch 10, training loss falls further while validation loss rises to 0.3020. The gap and fluctuations suggest that generalization should be monitored in longer runs, but the present evidence does not establish severe overfitting.

MLP’s macro-F1 is slightly higher at epoch 10 than at epoch 9. We still report epoch 9 because checkpoint selection is based on validation loss. Ten epochs were the baseline budget for M1; we have not tested whether a longer budget would be better.

Training metrics are accumulated while weights change between batches. Validation metrics use the fixed weights at the end of each epoch. This difference matters when comparing the curves. The full logs are in [Linear history](#) and [MLP history](#).

3.3 Confusion matrices and class-level errors

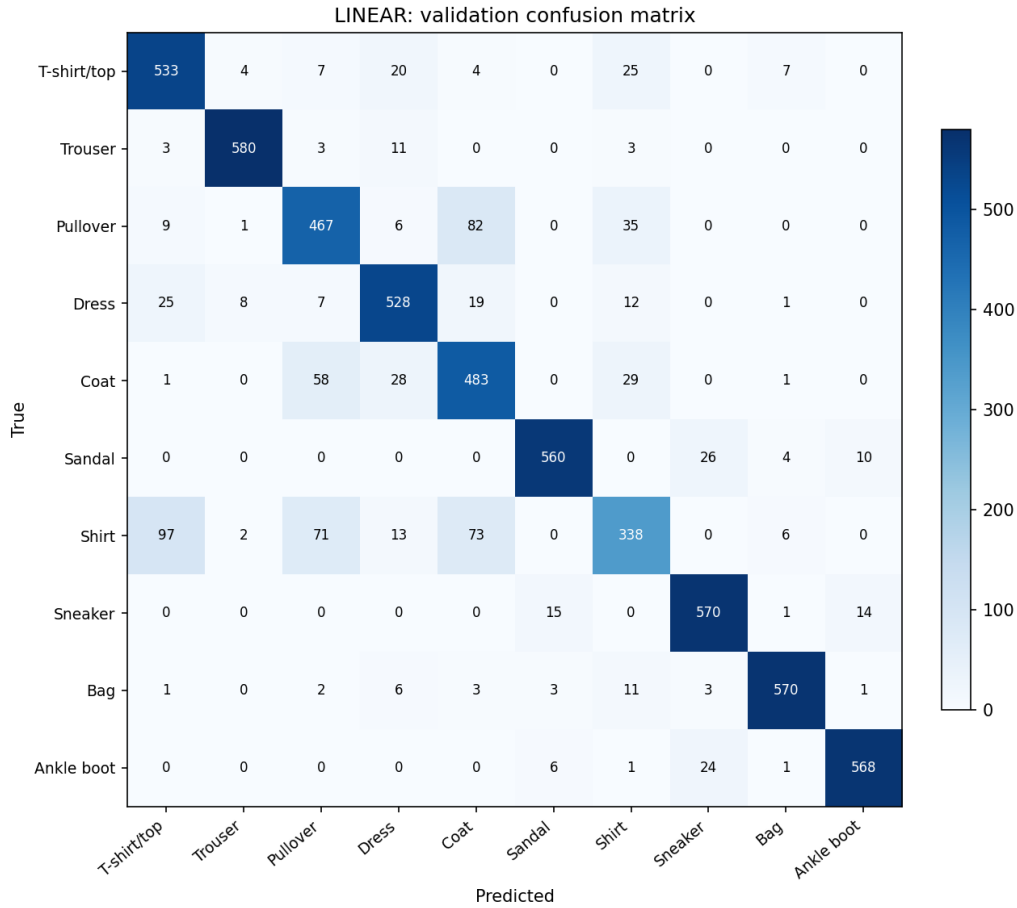


Figure 7: Linear confusion matrix on the validation set

The five most frequent Linear confusions are:

Table 5: The five most frequent Linear confusions.

True class	Predicted class	Images	Rate within true class
Shirt	T-shirt/top	97	16.17%
Pullover	Coat	82	13.67%
Shirt	Coat	73	12.17%
Shirt	Pullover	71	11.83%
Coat	Pullover	58	9.67%

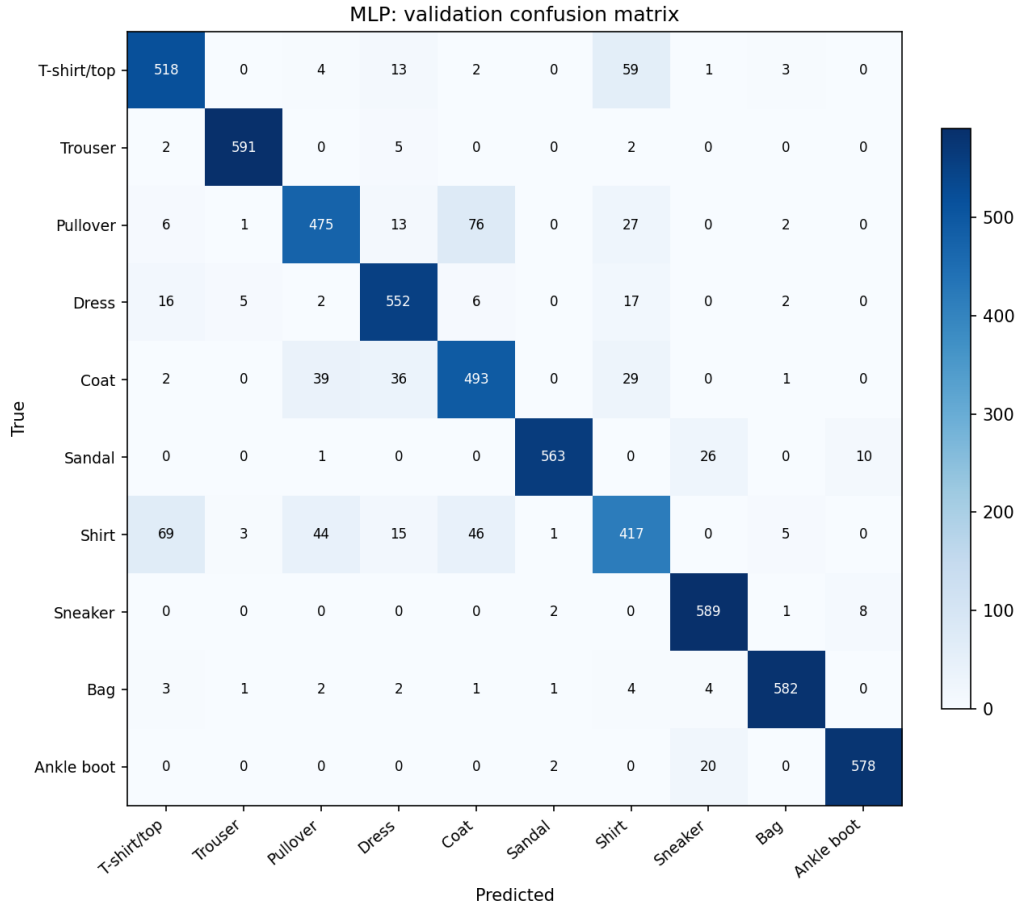


Figure 8: MLP confusion matrix on the validation set

The five most frequent MLP confusions are:

Table 6: The five most frequent MLP confusions.

True class	Predicted class	Images	Rate within true class
Pullover	Coat	76	12.67%
Shirt	T-shirt/top	69	11.50%
T-shirt/top	Shirt	59	9.83%
Shirt	Coat	46	7.67%
Shirt	Pullover	44	7.33%

MLP reduces Shirt → T-shirt/top errors from 97 to 69, Shirt → Coat from 73 to 46, Shirt → Pullover from 71 to 44, and Pullover → Coat from 82 to 76. Pullover → Coat

nevertheless remains MLP's largest confusion pair.

The following table summarizes all predictions by true class. Recall is the number of correct predictions for that class divided by 600.

Table 7: Errors and recall by true class on the full validation set.

True class (600 images each)	Linear errors	MLP errors	Linear recall	MLP recall
T-shirt/top	67	82	88.83%	86.33%
Trouser	20	9	96.67%	98.50%
Pullover	133	125	77.83%	79.17%
Dress	72	48	88.00%	92.00%
Coat	117	107	80.50%	82.17%
Sandal	40	37	93.33%	93.83%
Shirt	262	183	56.33%	69.50%
Sneaker	30	11	95.00%	98.17%
Bag	30	18	95.00%	97.00%
Ankle boot	32	22	94.67%	96.33%

Shirt has the most errors for both models, falling from 262 errors (43.67% of the class) to 183 (30.50%). T-shirt/top becomes slightly worse, with errors rising from 67 to 82. MLP's overall improvement therefore does not extend to every class. These counts come from the complete validation_predictions.csv files.

3.4 Correct and incorrect predictions

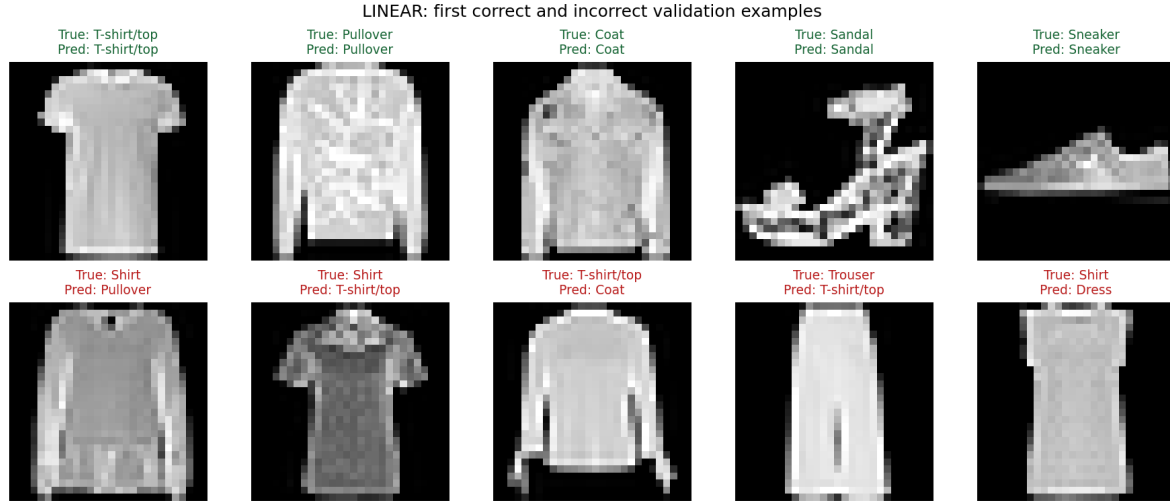


Figure 9: Linear: the first five correct and first five incorrect validation predictions

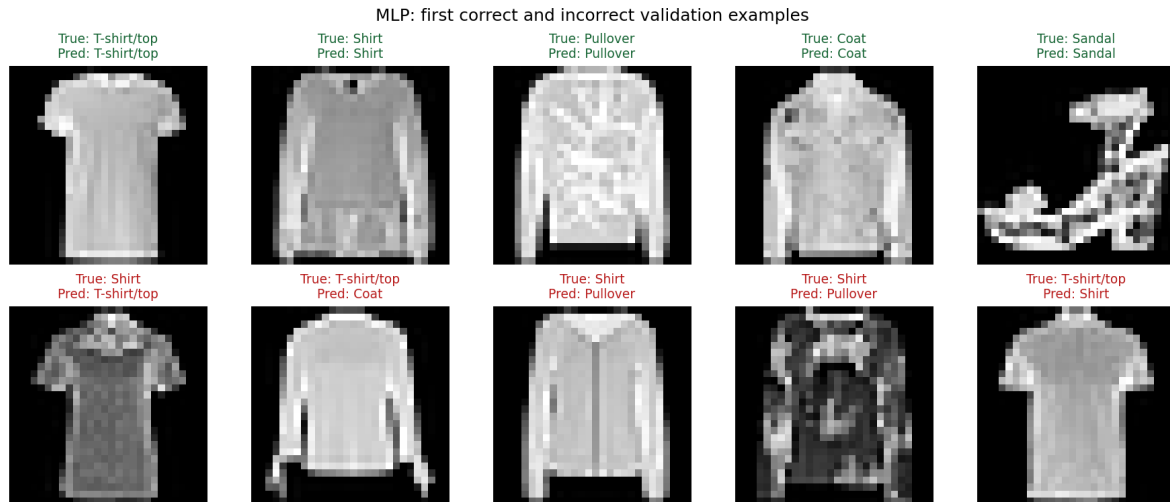


Figure 10: MLP: the first five correct and first five incorrect validation predictions

The top row of each figure shows the first five correct predictions; the bottom row shows the first five errors in validation order. These examples are selected separately for each model. They are not random samples, and matching columns do not necessarily show the same image.

Case A - original training index 18, true label Shirt. This image appears in the bottom-left position for Linear and in the second position of the top row for MLP. The

gray, long-sleeved garment has an outline similar to a pullover, while the collar and body details are small. Linear predicts Pullover; MLP correctly predicts Shirt. The example shows that the two models handle this image differently, but it does not identify the features MLP used.

Case B - index 164, true label Shirt. Both models predict T-shirt/top. The image has a dark body and short sleeves, with limited collar and torso detail at this resolution. Its T-shirt-like outline is a possible explanation for the error. It also belongs to the frequent Shirt $\rightarrow$ T-shirt/top confusion pair.

Case C - index 169, true label T-shirt/top. Both models predict Coat. The long sleeves and wide outline may make it resemble a coat. This is a visual interpretation of the example; it does not show that the original label is wrong or establish which features caused the prediction.

A useful next step is to test a CNN that models local 2D structure, alongside controlled regularization or augmentation experiments. Their effects have not been measured in M1.

3.5 Computational cost and timing scope

Table 8: Recorded training, validation and forward-pass timings.

Model	Training (s)	Validation (s)	Fit wall (s)	Forward / batch (ms)	Forward / image (ms)
LINEAR	64.62	7.05	71.68	0.039946	0.000312
MLP	64.04	6.97	71.03	0.137489	0.001074

Training time sums the training loops across 10 epochs. It includes batch loading, data transfer, forward passes, loss calculation, backpropagation, parameter updates and metric collection. It excludes validation, history/checkpoint writing, plotting and report generation. Validation time is measured separately.

Fit wall time covers the 10-epoch loop, including training, validation, logging and checkpoint writes. It excludes the initial data download and EDA, as well as final checkpoint evaluation and plotting. It therefore does not measure the entire notebook run.

Inference timing measures model forward passes in eval/inference_mode, with inputs already on the GPU. It uses batch size 128, 10 warm-up iterations and 50 timed iterations,



with CUDA synchronization around the timed section. Forward time per image is the batch time divided by 128. It excludes the DataLoader and data transfer and should not be treated as the latency of a single-image request.

Training times are close in this run, while MLP's forward time per image is about 3.44 times higher. Data loading, transfers and execution overhead can matter when models are small, but we have not profiled these components. Similar training times alone do not establish similar computational complexity.

3.6 Implementation issues

One notebook cell reads the wrong metric keys, `parameter_count` and `validation`, and prints `None`. The next cell uses the correct keys: `parameters`, `loss`, `accuracy` and `macro_f1`. This was a display issue in the summary cell; it did not affect training or the saved checkpoints.

The installation log also reports a compatibility warning between `numba 0.61.2` and `numpy 2.3.5`. The pipeline does not use `numba`, and training completes afterward. The supplied run does not document a full resolution of that environment conflict.

3.7 Reproduction and provenance

The original Colab run used a source ZIP. [Commit 5d7430a](#) archives the source files whose checksums match that run. It was created after training; the original `git_commit` field remains null. [COLAB_RUN.md](#) explains the source fingerprint and the later documentation changes.

To repeat the experiment on a Tesla T4 in Colab, select a T4 GPU runtime, then clone the repository and install the pinned dependencies as described in [README](#). Before training, check the device in a Python cell:

```
import torch
assert torch.cuda.is_available(), "CUDA is unavailable; select a GPU
↪ runtime."
print(torch.__version__)
print(torch.cuda.get_device_name(0))
```



For a hardware-matched rerun, the device name should identify a Tesla T4. T4 is the GPU model; cuda is the PyTorch device used to execute on it. From the repository directory in Colab, run these cells in order:

```
!python run_a1.py --stage all --device cuda --out outputs/a1_t4_rerun
```

```
!python run_a1.py --stage evaluate --device cuda --out outputs/a1_t4_rerun
```

In a terminal, use the same commands without the leading exclamation mark. The all stage runs EDA, trains both models and writes results. The evaluate stage reloads the checkpoints and evaluates them on validation. Use a new output directory for every rerun so that the original T4 evidence stays intact.

The recorded configuration is in [outputs/a1_t4/config.json](#). The default configs/a1.json selects CPU; `--device cuda` overrides that setting. The documentation, ‘a1/report.py’, and the explanatory pipeline diagram were updated after the original run. The model, data, EDA, and training implementations remain unchanged from the archived source. The model, data and training code still match the archived source.

Checkpoints: [Linear best.pt](#) and [MLP best.pt](#). Runtime and results may vary with hardware, software versions and nondeterminism.

3.8 Conclusions, limitations and next steps

M1 includes a working EDA stage, Dataset/DataLoader setup, a training and validation loop, and runnable Linear and MLP baselines. Under the reported T4 setup, MLP achieves higher validation accuracy and macro-F1, with more parameters and a higher forward-pass cost.

The current evidence is limited to one seed and one configuration. Model sizes are not matched, hyperparameters have not been searched, and the test set has not been evaluated. Image-content duplicates and generalization beyond Fashion-MNIST have not been checked. MNIST debugging and the CIFAR-10 extension were not run.

For the final milestone, we plan to add a custom CNN, an LSTM or GRU, and a Transformer. We will extend the comparisons and analysis, evaluate the test set after making validation-based choices, and prepare the final report, slides and video. These are planned tasks, not completed experiments.



AI Usage Disclosure

Our team used AI to develop ideas, review source code and improve the report. The reported experiments were run on Google Colab with an NVIDIA Tesla T4. Tools, scope and verification are documented in [AI_USAGE.md](#).

References

1. [Zalando Research, Fashion-MNIST: dataset description, format and license.](#)
2. [PyTorch Quickstart: Dataset/DataLoader, models and the optimization loop.](#)
3. Course Project Handbook CO3133, Semester 261, revision 14 September 2026; Sections 3, 4.2, 5, 7.1, 7.4 and Part II.
4. [Original Colab notebook supplied for this run.](#)
5. [Archived source matching the Colab fingerprint](#) and [saved experiment results](#).